

```
[1]: from __future__ import annotations
```

```
import hashlib
import json
import time
from typing import Any

print("imports ready")

imports ready
```

```
]: def sha256_hex(data: str) -> str:
    return hashlib.sha256(data.encode("utf-8")).hexdigest()

class Block:
    """Minimal block used by the chain. You may replace with your A2 Block."""

    def __init__(
        self,
        index: int,
        transactions: list[Any],
        previous_hash: str,
        nonce: int = 0,
        timestamp: float | None = None,
    ) -> None:
        self.index = index
        self.timestamp = time.time() if timestamp is None else timestamp
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.compute_hash()
```

```
    def compute_hash(self) -> str:
        payload = {
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce,
        }
        blob = json.dumps(payload, sort_keys=True, separators=(",", ":"))
        return sha256_hex(blob)

    def __repr__(self) -> str:
        return f"Block(index={self.index}, hash={self.hash[:12]}...)"

#stored hash matches recomputation
demo = Block(0, [{"note": "ping"}], previous_hash="0" * 64, nonce=0, timestamp=0.0)
assert demo.hash == demo.compute_hash()
assert len(demo.hash) == 64
assert all(c in "0123456789abcdef" for c in demo.hash)
print(demo)
GENESIS_PREV = "0" * 64
```

```
assert set(GENESIS_PREV) == {"0"}
print("GENESIS_PREV =", GENESIS_PREV[:8] + "...")
```

```
Block(index=0, hash=2aac41d6b854...)
GENESIS_PREV = 00000000...
```

```
[3]: class Blockchain:
    def __init__(self) -> None:
        self.chain: list[Block] = []
        self.chain.append(self.create_genesis_block())

    def create_genesis_block(self) -> Block:
        return Block(0, [{"note": "genesis"}], GENESIS_PREV, nonce=0, timestamp=0.0)

    def tip(self) -> Block:
        return self.chain[-1]

    def append_block(self, block: Block) -> None:
        """Append only if index, previous_hash, and self-hash checks pass."""
        tip = self.tip()
        if block.index != tip.index + 1:
            raise ValueError(f"bad index: expected {tip.index + 1}, got {block.index}")
        if block.previous_hash != tip.hash:
            raise ValueError("bad previous_hash: does not match chain tip")

        # recompute previous_hash = tip.hash
        if block.previous_hash != tip.hash:
            raise ValueError("bad previous_hash: does not match chain tip")
        if block.hash != block.compute_hash():
            raise ValueError("stored hash does not match recomputation")
        self.chain.append(block)

    def verify_chain(self, difficulty: int | None = None) -> bool:
        """Return True iff the full chain is internally consistent.

        If difficulty is None, every non-genesis block hash must start
        with that many hex zeros (optional A5 extension / L06 teaser).
        """
        if not self.chain:
            return False
        genesis = self.chain[0]
        if genesis.index != 0 or genesis.previous_hash != GENESIS_PREV:
            return False
        if genesis.hash != genesis.compute_hash():
            return False

        prefix = ("0" * difficulty) if difficulty is not None else None

        for i in range(1, len(self.chain)):
            cur, prev = self.chain[i], self.chain[i - 1]

            # for i in range(1, len(self.chain)):
            cur, prev = self.chain[i], self.chain[i - 1]
            if cur.index != i:
                return False
            if cur.hash != cur.compute_hash():
                return False
            if cur.previous_hash != prev.hash:
                return False
            if prefix is not None and not cur.hash.startswith(prefix):
                return False
            return True

    def __len__(self) -> int:
        return len(self.chain)

    def make_next_block(
        chain: Blockchain, transactions: list[Any], nonce: int = 0
    ) -> Block:
        """Build the next block pointing at the current tip (no mining yet)."""
        tip = chain.tip()
        return Block(tip.index + 1, transactions, tip.hash, nonce=nonce)
```

```
print("Blockchain API ready:", [m for m in dir(Blockchain) if not m.startswith("_")])

Blockchain API ready: ['append_block', 'create_genesis_block', 'tip', 'verify_chain']
```

```
[4]: bc = Blockchain()

assert len(bc) == 1
assert bc.tip().index == 0
assert bc.tip().previous_hash == GENESIS_PREV
assert bc.tip().hash == bc.tip().compute_hash()
assert bc.verify_chain() is True

print("length", len(bc))
print("tip ", bc.tip())
print("genesis txs:", bc.tip().transactions)

length 1
tip      Block(index=0, hash=aa7fed24e783...)
genesis txs: [{'note': 'genesis'}]
```

Question 2

```
5]: #Question 2

b1 = make_next_block(
    bc, [{"sender": "Alice", "recipient": "Bob", "amount": 10}]
)
bc.append_block(b1)

b2 = make_next_block(
    bc, [{"sender": "Bob", "recipient": "Carol", "amount": 4}]
)
bc.append_block(b2)

assert len(bc) == 3
assert bc.verify_chain() is True
assert bc.chain[1].previous_hash == bc.chain[0].hash
assert bc.chain[2].previous_hash == bc.chain[1].hash

print("verify", bc.verify_chain())
for blk in bc.chain:
    print(blk.index, blk.hash[:16] + "...", "txs", blk.transactions)
tip_before = bc.tip().hash
length_before = len(bc)
```

```
#Question 3
# Bad previous_hash (orphan / spliced block)
bad_link = Block(length_before, [{"x": 1}], previous_hash="deadbeef" * 8)
try:
    bc.append_block(bad_link)
    raise AssertionError("expected ValueError for bad previous_hash")
except ValueError as exc:
    print("rejected bad previous_hash:", exc)

# Bad index (gap / duplicate page number)
bad_index = Block(99, [{"x": 1}], previous_hash=tip_before)
try:
    bc.append_block(bad_index)
    raise AssertionError("expected ValueError for bad index")
except ValueError as exc:
    print("rejected bad index:", exc)

# Bad stored hash (claimed digest is a lie)
liar = make_next_block(bc, [{"x": 1}])
liar.hash = "ff" * 32
try:
    bc.append_block(liar)
    raise AssertionError("expected ValueError for bad stored hash")
except ValueError as exc:
```

```

    print("rejected bad stored hash:", exc)

assert len(bc) == length_before
assert bc.tip().hash == tip_before
assert bc.verify_chain() is True
print("chain unchanged - still valid, length", len(bc))

verify True
0 aa7fed24e78336bd... txs [{'note': 'genesis'}]
1 7874691f09b4f73e... txs [{'sender': 'Alice', 'recipient': 'Bob', 'amount': 10}]
2 ae589948c2a02d01... txs [{'sender': 'Bob', 'recipient': 'Carol', 'amount': 4}]
rejected bad previous_hash: bad previous_hash: does not match chain tip
rejected bad index: bad index: expected 3, got 99
rejected bad stored hash: stored hash does not match recomputation
chain unchanged - still valid, length 3

```

```

# Fresh chain so the demo is self-contained and reproducible
ledger = Blockchain()
ledger.append_block(
    make_next_block(ledger, [{"sender": "Alice", "recipient": "Bob", "amount": 10}])
)
ledger.append_block(
    make_next_block(ledger, [{"sender": "Bob", "recipient": "Carol", "amount": 4}])
)
assert ledger.verify_chain() is True

```

```

)
assert ledger.verify_chain() is True
print("valid chain:", ledger.verify_chain(), "length", len(ledger))

old_hash_1 = ledger.chain[1].hash
old_prev_2 = ledger.chain[2].previous_hash
assert old_prev_2 == old_hash_1

# Mutate in place
ledger.chain[1].transactions[0]["amount"] = 999
print("after mutation, verify =", ledger.verify_chain())
assert ledger.verify_chain() is False
assert ledger.chain[1].hash == old_hash_1 # stored digest not updated yet
assert ledger.chain[1].hash != ledger.chain[1].compute_hash() # pillar 2 fails
print("pillar 2: stored hash != recomputation for block 1")

valid chain: True length 3
after mutation, verify = False
pillar 2: stored hash != recomputation for block 1

```

```

]: # "Repair" only block 1's stored hash
ledger.chain[1].hash = ledger.chain[1].compute_hash()
print("after re-hash block 1 only, verify =", ledger.verify_chain())
assert ledger.verify_chain() is False

```

```

# Diagnose pillar 3: block 2 still stores the old previous_hash
assert ledger.chain[1].hash == ledger.chain[1].compute_hash() # block 1 now self-consistent
assert ledger.chain[2].previous_hash == old_hash_1
assert ledger.chain[2].previous_hash != ledger.chain[1].hash
print("pillar 3: block 2.previous_hash is stale")
print("  block 2.previous_hash:", ledger.chain[2].previous_hash[:16] + "...")
print("  block 1.hash now:      ", ledger.chain[1].hash[:16] + "...")

after re-hash block 1 only, verify = False
pillar 3: block 2.previous_hash is stale
block 2.previous_hash: 757fe652fc0f19a0...
block 1.hash now:      91fd3cde4d9927b8...

```

```

0]: # Rebuild Links from the mutated block onward
for i in range(1, len(ledger.chain)):
    if i > 1:
        ledger.chain[i].previous_hash = ledger.chain[i - 1].hash
        ledger.chain[i].hash = ledger.chain[i].compute_hash()

print("after full suffix rewrite, verify =", ledger.verify_chain())
assert ledger.verify_chain() is True
assert ledger.chain[1].transactions[0]["amount"] == 999 # fraud still in the data!
print("Without PoW, rewrite is cheap - the fraudulent amount is now 'valid' history.")
print("Next week we add work so suffix rewrite costs nonce search.")

```

after full suffix rewrite, verify = True
 Without POW, rewrite is cheap - the fraudulent amount is now 'valid' history.
 Next week we add work so suffix rewrite costs nonce search.

```
11]: pow_chain = Blockchain()
    blk = make_next_block(pow_chain, [{"amount": 1}])

    # Mine manually to difficulty
    nonce = 0
    while True:
        blk.nonce = nonce
        digest = blk.compute_hash()
        if digest.startswith("00"):
            blk.hash = digest
            break
        nonce += 1

    pow_chain.append_block(blk)
    assert pow_chain.verify_chain() is True
    assert pow_chain.verify_chain(difficulty=2) is True
    print(f"mined nonce={blk.nonce}, hash={blk.hash[:16]}...")
    print("verify(difficulty=2):", pow_chain.verify_chain(difficulty=2))

# Force a non-compliant hash - difficulty check must fail
pow_chain.chain[1].nonce = 0
pow_chain.chain[1].hash = pow_chain.chain[1].compute_hash()
if not pow_chain.chain[1].hash.startswith("00"):
    assert pow_chain.verify_chain(difficulty=2) is False
    print("after nonce reset, verify(difficulty=2) =", pow_chain.verify_chain(difficulty=2))
else:
    # Extremely unlikely at difficulty 2 with nonce=0, but keep the notebook robust
    print("nonce=0 happened to meet difficulty - skip negative assert")

mined nonce=366, hash=00820f02d2929362...
verify(difficulty=2): True
after nonce reset, verify(difficulty=2) = False
```

PART B

```
1]: #Part B

def dhash(data: bytes) -> bytes:
    """Double SHA-256, as used in Bitcoin."""
    return hashlib.sha256(hashlib.sha256(data).digest()).digest()

transactions = [
    "Alice pays Bob 10 BTC",
    "Bob pays Sipho 28 BTC",
    "Sipho pays Mba 15 BTC",
    "Mba pays Limane 3 BTC",
]

# Leaf hashes
leaf_hashes = [dhash(tx.encode('utf-8')) for tx in transactions]
HA, HB, HC, HD = leaf_hashes

# pair and hash
HAB = dhash(HA + HB)
HCD = dhash(HC + HD)

# final pairing
ROOT = dhash(HAB + HCD)

# final pairing
ROOT = dhash(HAB + HCD)

for name, val in [("HA", HA), ("HB", HB), ("HC", HC), ("HD", HD),
                  ("HAB", HAB), ("HCD", HCD), ("ROOT", ROOT)]:
    print(f"{name}: {val.hex()}")

HA: d375600e273d9728a0a0445b97f84d59ca3ad587d1e8c113fcdcbc064bdd37d34
HB: 27cbac03b1cb50dd6d43996c72b7a7d613c40adc880d88ebfd29e9b57b9c0706
HC: 33a66da93aa00399fb4107a6ddc55e368faf84f28d9c2d22878c610f5ecf22af
HD: fb349d282ad7ba96e343d889e2354f764e19f017f5e40b09d9194dd36a050ae3
HAB: 1a78b56b8df747b1a3ef9e140817896ba33cc2ced8359f2087c95905eb8aa4a4
HCD: 3c6de7c2e619c57426572491caef5e65c8aa20a966cbf1516076519bdad4655e
ROOT: 2d0b1d3df2e499a813e783405b182e6bb2d961ca0ae7dee5a47baf4a7a941edf
```